

Final Project MATH 4553 : Spring 2019

Optimization of a Repairable Multi-State System

Md Yeam Hossain
Saadbin Khan

8 May 2019

Abstract

Repairable multi-state device is a good platform to implement quadratic programming to determine the optimal control input. We approach this well-known optimal control problem both analytically and using Lagrange method. Backward Euler's method is used to discretize the system of ODEs associated with this problem. With reasonably fine convergence criteria for the Lagrange method, the solution is found to be very close to the analytical solution.

1 Problems Description

The mathematical model of a repairable multi-state device with arbitrarily distributed repair time has been introduced in Chung (1981). Here we consider the simplified model, which can be described by the ordinary differential equation as follows.

$$\begin{aligned}\frac{dp_0(t)}{dt} &= -\lambda_0 p_0(t) + \mu_1 p_1(t) \\ \frac{dp_1(t)}{dt} &= -\mu_1 p_1(t) + \lambda_0 p_0(t)\end{aligned}\tag{1}$$

where,

$t \in [0, T_0]$ and the initial conditions are:

$$p_0(0) = 1 \text{ and } p_1(0) = 0\tag{2}$$

Here:

- (1) $p_0(t)$ Probability that the device is in good mode 0 at time t
- (2) $p_1(t)$ Probability that the failed device is in failure mode 1 at time t
- (3) $\lambda_0 = 0.3$ Constant failure rate of the device for failure mode 1
- (4) $\mu_1 = 0.65$ Constant repair rate when the device is in state 1

The following assumptions are associated with the device:

- (1) The failure rates are constant
- (2) All failures are statistically independent
- (3) There is only one mode of failure and 0 implies the good state
- (4) Repair is to like-new and it does not cause damage to any other part of the system
- (5) Transitions are permitted only between states 0 and 1
- (6) The repair process begins soon after the device is in failure state
- (7) The repaired device is as good as new
- (8) No further failure can occur when the device has been down

For this project we have to develop an optimal control design for the reparable system. In order to maintain the system with desired probability the following optimization problem is considered which is derived from (1):

$$\begin{aligned}\frac{dp_0(t)}{dt} &= -0.25p_0(t) + 0.75p_1(t) + u(t) \\ \frac{dp_1(t)}{dt} &= -0.75p_1(t) + 0.25p_0(t) - u(t)\end{aligned}\tag{3}$$

where function $u(t)$ is the control input, which represents the system maintenance, through which the system can be steered to the desired probability distribution over the time interval $[0, T_0]$. We have to consider the following minimization problem:

$$J(p_0(t); p_1(t); u(t)) = \|p_0(t) - p_0^*(t)\|_2^2 + \|p_1(t) - p_1^*(t)\|_2^2 + \|u(t)\|_2^2\tag{4}$$

subject to the system in (3) with initial conditions in (2). Our goal is to minimize the objective function using optimal control input $u(t)$. Here, the time interval is set as $[0, T_0]$ as $[0, 5]$. We also have the assumption that the desired probability of the system in good state is $p_0^* = 90\%$, and the desired probability of the system in failure state is $p_1^* = 10\%$. Our goal is to obtain an optimal maintenance $u(t)$, that will minimize the objective function (4).

2 Methodology

The system (3) can be rewritten in matrix-vector form:

$$\frac{d}{dt} \begin{bmatrix} p_0(t) \\ p_1(t) \end{bmatrix} = A \begin{bmatrix} p_0(t) \\ p_1(t) \end{bmatrix} + Bu(t),\tag{5}$$

where

$$A = \begin{bmatrix} -0.25 & 0.75 \\ 0.25 & -0.75 \end{bmatrix} \text{ and } B = \begin{bmatrix} 1 \\ -1 \end{bmatrix}\tag{6}$$

Next, we will discretize this system of ODEs using finite difference approximation.

2.1 Discretization of ODEs

By using the backward Euler's method we can discretize the linear ordinary differential equations and reduce the controlled system (5) of algebraic equations as:

$$\begin{bmatrix} \frac{p_0(t_{i+1})-p_0(t_i)}{\Delta t} \\ \frac{p_1(t_{i+1})-p_1(t_i)}{\Delta t} \end{bmatrix} = A \begin{bmatrix} p_0(t_{i+1}) \\ p_1(t_{i+1}) \end{bmatrix} + Bu(t_{i+1}), \quad (7)$$

Here we divide the time interval $[0, T_0] = [0, 5]$ into 100 equal subintervals $[t_{i-1}, t_i]$, where $i = 0, 1, \dots, 99$, and set $\Delta t = \frac{1}{20}$. Then $t_i = i\Delta t$ and

$$0 = t_0 < t_1 < \dots < t_i < \dots < t_{100} = 5$$

Denoted by $p_{0,i} = p_0(t_i)$, $p_{1,i} = p_1(t_i)$ and $u_i = u(t_i)$, system (7) can be rewritten as:

$$\begin{bmatrix} \frac{p_{0,i+1}-p_{0,i}}{\Delta t} \\ \frac{p_{1,i+1}-p_{1,i}}{\Delta t} \end{bmatrix} = A \begin{bmatrix} p_{0,i+1} \\ p_{1,i+1} \end{bmatrix} + Bu_{i+1}, \quad (8)$$

for $i = 0, 1, \dots, 99$, where according to the initial conditions given by (2):

$$p_{0,0} = 1, p_{1,0} = 0$$

Additionally, we let

$$P_0 = [p_{0,1}, p_{0,2}, \dots, p_{0,100}]^T, \quad P_1 = [p_{1,1}, p_{1,2}, \dots, p_{1,100}]^T$$

and

$$\mathbf{u} = [u_1, u_2, \dots, u_{100}]^T,$$

then the objective function (4) becomes

$$J(P_0; P_1; \mathbf{u}) = \left\| \begin{bmatrix} P_0 \\ P_1 \end{bmatrix} - \begin{bmatrix} P_0^* \\ P_1^* \end{bmatrix} \right\|_2^2 + \|\mathbf{u}\|_2^2, \quad (9)$$

where

$$P_0^* = [90\%, 90\%, \dots, 90\%]_{100 \times 1}^T, \quad P_1^* = [10\%, 10\%, \dots, 10\%]_{100 \times 1}^T.$$

2.2 Building the System

To start with, z is defined as:

$$z = [P_0 - P_0^* \quad P_1 - P_1^* \quad \mathbf{u}]^T \quad (10)$$

and x as

$$x = [P_0 \quad P_1 \quad \mathbf{u}]^T \quad (11)$$

Using the following equations we enforce the cost function (4) by solving a constrained system of z to solve for the $Az = \tilde{b}$:

$$x = z + [P_0^* \quad P_1^* \quad 0]^T$$

$$Ax = b \quad (12)$$

$$Az + A \begin{bmatrix} P_0^* & P_1^* & 0 \end{bmatrix}^T = b \quad (13)$$

$$Az = b - A \begin{bmatrix} P_0^* & P_1^* & 0 \end{bmatrix}^T = \tilde{b} \quad (14)$$

For $i = 0, 1, \dots, 99$, we find the values for each row of the fundamental block of matrices that will be incorporated into the big matrix representing the whole system (8)

$$\left(\frac{I}{\Delta t} - A\right) \begin{bmatrix} p_{0,i+1} \\ p_{1,i+1} \end{bmatrix} = \frac{1}{\Delta t} \begin{bmatrix} p_{0,i} \\ p_{1,i} \end{bmatrix} + Bu_{i+1}, \quad (15)$$

Which is then reduced to the form:

$$\begin{bmatrix} p_{0,i+1} \\ p_{1,i+1} \end{bmatrix} = \left(\frac{I}{\Delta t} - A\right)^{-1} \frac{1}{\Delta t} \begin{bmatrix} p_{0,i} \\ p_{1,i} \end{bmatrix} + \left(\frac{I}{\Delta t} - A\right)^{-1} Bu_{i+1}. \quad (16)$$

Substituting the terms A and B , the generalized scheme takes the form:

$$\begin{bmatrix} p_{0,i+1} \\ p_{1,i+1} \end{bmatrix} = \begin{bmatrix} \alpha & \beta \\ \sigma & \phi \end{bmatrix} \begin{bmatrix} p_{0,i} \\ p_{1,i} \end{bmatrix} + \begin{bmatrix} \gamma \\ \tau \end{bmatrix} u_{i+1} \quad (17)$$

Using the first row of elements, we can develop A such that it becomes a sparse matrix in the form of:

$$A = \begin{bmatrix} a & b & e \\ c & d & f \end{bmatrix} \quad (18)$$

where,

$$\begin{aligned} a &= \begin{bmatrix} 1 & 0 & 0 & \dots & 0 \\ -\alpha & 1 & 0 & \dots & 0 \\ 0 & -\alpha & 1 & \dots & 0 \\ \vdots & \vdots & & \ddots & \vdots \\ 0 & 0 & \dots & -\alpha & 1 \end{bmatrix} \\ b &= \begin{bmatrix} 0 & 0 & 0 & \dots & 0 \\ -\beta & 0 & 0 & \dots & 0 \\ 0 & -\beta & 0 & \dots & 0 \\ \vdots & \vdots & & \ddots & \vdots \\ 0 & 0 & \dots & -\beta & 0 \end{bmatrix} \\ e &= \begin{bmatrix} -\gamma & 0 & 0 & \dots & 0 \\ 0 & -\gamma & 0 & \dots & 0 \\ 0 & 0 & -\gamma & \dots & 0 \\ \vdots & \vdots & & \ddots & \vdots \\ 0 & 0 & \dots & 0 & -\gamma \end{bmatrix} \\ c &= \begin{bmatrix} 0 & 0 & 0 & \dots & 0 \\ -\sigma & 0 & 0 & \dots & 0 \\ 0 & -\sigma & 0 & \dots & 0 \\ \vdots & \vdots & & \ddots & \vdots \\ 0 & 0 & \dots & -\sigma & 0 \end{bmatrix} \end{aligned}$$

$$d = \begin{bmatrix} 1 & 0 & 0 & \cdots & 0 \\ -\phi & 1 & 0 & \cdots & 0 \\ 0 & -\phi & 1 & \cdots & 0 \\ \vdots & \vdots & & \ddots & \vdots \\ 0 & 0 & \cdots & -\phi & 1 \end{bmatrix}$$

$$f = \begin{bmatrix} -\tau & 0 & 0 & \cdots & 0 \\ 0 & -\tau & 0 & \cdots & 0 \\ 0 & 0 & -\tau & \cdots & 0 \\ \vdots & \vdots & & \ddots & \vdots \\ 0 & 0 & \cdots & 0 & -\tau \end{bmatrix}$$

Next, we find b as

$$b = \left[\alpha \quad 0 \quad \cdots \quad 0 \mid \sigma \quad 0 \quad \cdots \quad 0 \right]_{200 \times 1}^T$$

Then, to convert the system into quadratic form completely we derive Q by reducing the cost function (9) to a Hessian form. Let $P = [P_0 P_1]^T$. By taking the dot product of the L2 norms, the cost function reduces to;

$$\begin{aligned} J(P; \mathbf{u}) &= \frac{1}{2} \langle P - P^*, P - P^* \rangle + \frac{1}{2} \langle \mathbf{u}, \mathbf{u} \rangle \\ &= \frac{1}{2} (P^T P - 2P^T P^* + P^{*T} P^*) + \frac{1}{2} \mathbf{u}^T \mathbf{u}. \end{aligned} \quad (19)$$

Now the problem is reduces to a diagonal matrix, $Q = 2 * I$ with respect to vector z . Following the relation below, an optimal solution can be obtained from these matrices.

$$z^* = Q^{-1} A^T (A Q^{-1} A^T)^{-1} \tilde{b} \quad (20)$$

2.3 Lagrange's Algorithm

The Lagrange function can be defined as follows-

$$L(z, \lambda) = \frac{1}{2} z^T Q z + \lambda^T (A z - \tilde{b}) \quad (21)$$

where,

λ is Lagrange multiplier.

The Lagrange conditions are:

$$\begin{aligned} \nabla_z L(z, \lambda) &= Q z + A^T \lambda \\ \nabla_\lambda L(z, \lambda) &= A z - \tilde{b}. \end{aligned} \quad (22)$$

The Lagrange Algorithm is developed by using the Lagrange conditions. We initialize the algorithm with an initial guess for the system state z_0 and Lagrange multipliers λ_0 . We set dampening constants α and β to a sufficiently small size to ensure the algorithm smoothly converges to a solution after N iterations:

$$\begin{aligned} z_{k+1} &= z_k - \alpha (Q z_k + A^T \lambda_k) \\ \lambda_{k+1} &= \lambda_k - \beta (A z_k - \tilde{b}) \end{aligned} \quad (23)$$

We utilize the following stopping criteria for algorithm (23)

$$\left\| \frac{z_{k+1} - z_k}{z_k} \right\|_2 < 10^{-6}, \quad (24)$$

3 Results

In figure 1 both the analytical and numerical solution is presented. Because of the fact that our convergence criteria ϵ is as small as 10^{-6} , the solutions are pretty close to each other. If we select a little more lenient convergence criteria, then the deviations becomes visually distinguishable. This shows that the solutions closely correlate with each other.

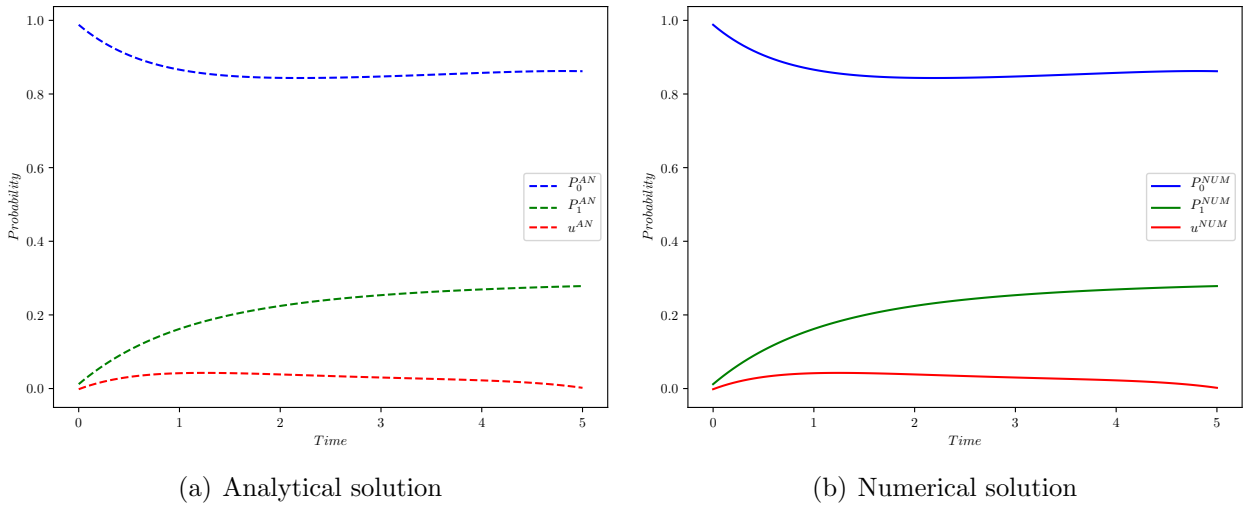


Figure 1: Solution of the optimization problem (a) Analytically, (b) using Lagrange Method

4 Observation and Conclusions

The deviation between the analytical and numerical solution approaches to be very close to zero with increasing convergence criteria. This is evidently seen from an error analysis between the two solution in Figure 2.

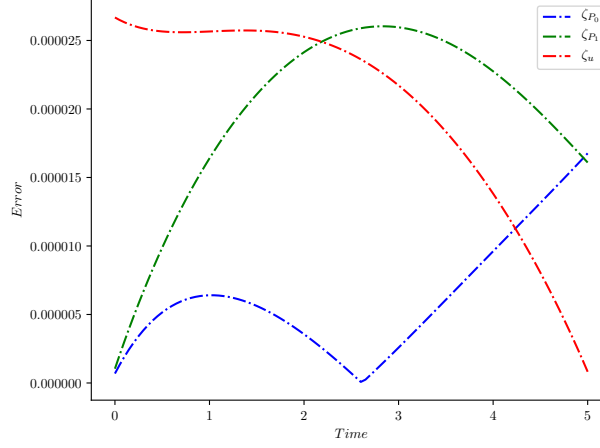


Figure 2: Error between analytical and numerical solution

However, the computational cost is higher as it takes more time to get to the correct solution if convergence criteria is very fine.

Appendix

Listing 1: Optimal control solver for 1-D convection-diffusion equation.

```
##### Final Project #####
##### MATH 4453 #####
##### SAADBIN KHAN & YEAM HOSSAIN #####
##### Optimization of a Repairable Multi-state System #####
```

```
import numpy as np
from numpy.linalg import inv
import matplotlib.pyplot as plt

plt.rc('font',**{'family':'sans-serif','sans-serif':['Helvetica']})
plt.rc('text', usetex=True)
size1 = 65 # 1*3 figure font
size2 = 20 # pads
size3 = 30
lwtdh1 = 8
```

SET UP

```

# inputs
nmode = 2 # no of modes
ncont = 1 # no of control input
p0st = 0.9 # desired probability of being in good state
p1st = 0.1 # desired probability of being in failure state
tst, ten = [0.0, 5.0] # time interval
nt = 101 # total no of timesnap
dt = (ten-tst)/(nt-1) # timestep
t = dt*np.arange(nt) # time vector

A = np.array([[ -0.25, 0.75], [0.25, -0.75]]) # coefficient matrix
B = np.array([[1], [-1]]) # coefficient vector associated with control input
I = np.eye(nmode) # identity matrix

# matrix and vector in the generalized system
coremat = inv(I/dt-A)/dt
corevec = inv(I/dt-A)*B

# tridiagonal matrix elements in the expanded system
a,b = coremat[0,:]
c,d = coremat[1,:]
e,f = corevec[:,0]

# a generalized function to build tridiagonal matrices
def tridiag(_a, _b, _c, _n):
    _mat = np.zeros([_n,_n])
    for _i in range(_n):
        for _j in range(_n):
            if(_j==_i): _mat[_i,_j] = _b
            if(_j==_i-1): _mat[_i,_j] = _a
            if(_j==_i+1): _mat[_i,_j] = _c
    return _mat

# building individual tridiagonal blocks
block_a = tridiag(-a, 1, 0, nt)
block_b = tridiag(-b, 0, 0, nt)
block_c = tridiag(-c, 0, 0, nt)
block_d = tridiag(-d, 1, 0, nt)
block_e = tridiag(0, -e, 0, nt)
block_f = tridiag(0, -f, 0, nt)

# building the sparse matrix for the time varying system using block matrices
bigmatA = np.block([[block_a, block_b, block_e],[block_c, block_d, block_f]])

```

```

# building the sparse vector associated with the time varying control input
vec_a = np.zeros([nt,1])
vec_a[0] = a
vec_c = np.zeros([nt,1])
vec_c[0] = c
bigvecb = np.block([vec_a.T, vec_c.T]).T

# building the expected probability vector
Pst = np.block([np.zeros([nt,1]).T+p0st, np.zeros([nt,1]).T+p1st, np.zeros([nt

# building diagonal matrix Q for the quadratic form
Q = 2*np.eye(nt*(nmode+ncont))

##### ANALYTICAL SOLUTION #####

btilda = bigvecb - np.matmul(bigmatA, Pst)
prod1 = np.matmul(inv(Q), bigmatA.T)
prod2 = np.matmul(bigmatA, prod1)
prod3 = np.matmul(bigmatA.T, inv(prod2))
prod4 = np.matmul(inv(Q), prod3)
zopt_an = np.matmul(prod4, btilda)

# solution
p0_an = zopt_an[:nt]+p0st
p1_an = zopt_an[nt:2*nt]+p1st
u_an = zopt_an[2*nt:]

##### NUMERICAL SOLUTION #####

eps = 1e-6 # acceptable tolerance

# initial values
z = np.ones([(nmode+ncont)*nt,1])
zold = np.copy(z)
lam = 0.1+np.zeros([nmode*nt,1]) # lagrange multiplier
alpha = 0.02 # dampening constant for cost function
beta = 0.05 # dampening constant for multiplier

# Lagrange loop
errnorm = 10
count = 0
while errnorm>eps:
    count = count+1
    print('Iteration no:', errnorm, count)
    z = z - alpha*(np.matmul(Q, zold)+np.matmul(bigmatA.T, lam)) # reduced cost

```

```

    lam = lam + beta*(np.matmul(bigmatA,zold)-btilda) # updated multiplier
    errnorm = (np.sum(abs((z-zold)/zold)**2))*0.5 # l2 error norm
    zold = np.copy(z)

# solution
p0_num = z[:nt]+p0st
p1_num = z[nt:2*nt]+p1st
u_num = z[2*nt:]

##### PLOTTING #####

fig,ax = plt.subplots()
plt.plot(t,p0_an,'b—', label=r'$P_{0}^{\{AN\}}$')
plt.plot(t,p1_an,'g—', label=r'$P_{1}^{\{AN\}}$')
plt.plot(t,u_an,'r—', label=r'$u^{\{AN\}}$')
plt.legend()
plt.xlabel(r'$Time$')
plt.ylabel(r'$Probability$')
plt.tick_params(axis='both')
plt.tight_layout()
plt.savefig('solution_analytical.pdf', bbox_inches='tight', dpi=200)

fig,ax = plt.subplots()
plt.plot(t,p0_num,'b—', label=r'$P_{0}^{\{NUM\}}$')
plt.plot(t,p1_num,'g—', label=r'$P_{1}^{\{NUM\}}$')
plt.plot(t,u_num,'r—', label=r'$u^{\{NUM\}}$')
plt.legend()
plt.xlabel(r'$Time$')
plt.ylabel(r'$Probability$')
plt.tick_params(axis='both')
plt.tight_layout()
plt.savefig('solution_numerical.pdf', bbox_inches='tight', dpi=200)

fig,ax = plt.subplots()
plt.plot(t,p0_num,'b—', label=r'$P_{0}^{\{NUM\}}$')
plt.plot(t,p1_num,'g—', label=r'$P_{1}^{\{NUM\}}$')
plt.plot(t,u_num,'r—', label=r'$u^{\{NUM\}}$')
plt.plot(t,p0_an,'b—', label=r'$P_{0}^{\{AN\}}$')
plt.plot(t,p1_an,'g—', label=r'$P_{1}^{\{AN\}}$')
plt.plot(t,u_an,'r—', label=r'$u^{\{AN\}}$')
plt.legend(loc='upper_center', bbox_to_anchor=(0.5,1.1), ncol=6)
plt.xlabel(r'$Time$')
plt.ylabel(r'$Probability$')
plt.tick_params(axis='both')
plt.tight_layout()
plt.savefig('solution_all.pdf', bbox_inches='tight', dpi=200)

```

```

fig,ax = plt.subplots()
plt.plot(t,abs(p0_num-p0_an),'b-.', label=r '$\zeta_{P_0}$')
plt.plot(t,abs(p1_num-p1_an),'g-.', label=r '$\zeta_{P_1}$')
plt.plot(t,abs(u_num-u_an),'r-.', label=r '$\zeta_u$')
plt.legend()
plt.xlabel(r '$Time$')
plt.ylabel(r '$Error$')
plt.tick_params(axis='both')
plt.tight_layout()
plt.savefig('solution_error.pdf', bbox_inches='tight', dpi=200)

```